MERN Expense Tracker: Starter Guide

What you'll build: users register and log in, add income/expense transactions, delete them, and see their balance. Stack: MongoDB, Express, React (Vite), Node.

Rough plan (3–4 days): Day 1 backend setup + auth. Day 2 transactions API. Day 3 React UI. Day 4 polish + deploy.

---

1. Prerequisites

- Node.js 18+ (`node -v`)
- VS Code, Git, Postman (or Thunder Client)
- Free MongoDB Atlas account (create a cluster, a DB user, allow IP `0.0.0.0/0`, copy the connection string)

2. Folder structure

```
expense-tracker/
├── server/
│   ├── models/        User.js, Transaction.js
│   ├── routes/        auth.js, transactions.js
│   ├── middleware/    auth.js
│   ├── server.js
│   └── .env
└── client/            (Vite React app)
    └── src/           App.jsx, api.js, pages/
```

3. Backend setup

```
mkdir expense-tracker && cd expense-tracker
mkdir server && cd server
npm init -y
npm i express mongoose cors dotenv bcryptjs jsonwebtoken
npm i -D nodemon
```

In `package.json` add scripts: `"dev": "nodemon server.js"`.

.env

```
PORT=5000
MONGO_URI=your_atlas_connection_string
JWT_SECRET=any_long_random_string
```

Add `.env` and `node_modules` to `.gitignore`.

server.js

```
require('dotenv').config();
```

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', require('./routes/auth'));
app.use('/api/transactions', require('./routes/transactions'));

mongoose.connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT, () => console.log('Server running')))
  .catch(err => console.error(err));
```

models/User.js

```
const mongoose = require('mongoose');
module.exports = mongoose.model('User', new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
}));
```

models/Transaction.js

```
const mongoose = require('mongoose');
module.exports = mongoose.model('Transaction', new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  amount: { type: Number, required: true },
  type: { type: String, enum: ['income', 'expense'], required: true },
  category: { type: String, default: 'General' },
  date: { type: Date, default: Date.now }
}));
```

middleware/auth.js

```
const jwt = require('jsonwebtoken');
module.exports = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ msg: 'No token' });
  try {
    req.userId = jwt.verify(token, process.env.JWT_SECRET).id;
    next();
  } catch {
    res.status(401).json({ msg: 'Invalid token' });
  }
};
```

routes/auth.js

```
const router = require('express').Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const sign = id => jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: '7d' });

router.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;
    if (await User.findOne({ email })) return res.status(400).json({ msg: 'Email already used' });
    const user = await User.create({ name, email, password: await bcrypt.hash(password, 10) });
    res.json({ token: sign(user._id), name: user.name });
  } catch (e) { res.status(500).json({ msg: e.message }); }
});

router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user || !(await bcrypt.compare(password, user.password)))
      return res.status(400).json({ msg: 'Invalid credentials' });
    res.json({ token: sign(user._id), name: user.name });
  } catch (e) { res.status(500).json({ msg: e.message }); }
});

module.exports = router;
```

routes/transactions.js

```
const router = require('express').Router();
const auth = require('../middleware/auth');
const Transaction = require('../models/Transaction');

router.use(auth);

router.get('/', async (req, res) =>
  res.json(await Transaction.find({ user: req.userId }).sort({ date: -1 })));

router.post('/', async (req, res) => {
  const t = await Transaction.create({ ...req.body, user: req.userId });
  res.json(t);
});

router.delete('/:id', async (req, res) => {
  await Transaction.findOneAndDelete({ _id: req.params.id, user: req.userId });
  res.json({ msg: 'Deleted' });
});

module.exports = router;
```

Test it with `npm run dev`, then in Postman: `POST /api/auth/register`, copy the token, and call `POST`

`/api/transactions` with header `Authorization: Bearer <token>`.

4. Frontend setup

From the project root:

```
npm create vite@latest client -- --template react
cd client
npm i axios react-router-dom
npm run dev
```

src/api.js

```
import axios from 'axios';
const api = axios.create({ baseURL: 'http://localhost:5000/api' });
api.interceptors.request.use(cfg => {
  const token = localStorage.getItem('token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});
export default api;
```

src/App.jsx (routing)

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Auth from './pages/Auth';
import Dashboard from './pages/Dashboard';

const Private = ({ children }) =>
  localStorage.getItem('token') ? children : <Navigate to="/login" />;

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login" element={<Auth mode="login" />} />
        <Route path="/register" element={<Auth mode="register" />} />
        <Route path="/" element={<Private><Dashboard /></Private>} />
      </Routes>
    </BrowserRouter>
  );
}
```

src/pages/Auth.jsx

```
import { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import api from '../api';

export default function Auth({ mode }) {
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [error, setError] = useState('');
```

```
  const nav = useNavigate();

  const submit = async () => {
    try {
      const { data } = await api.post(`/auth/${mode}`, form);
      localStorage.setItem('token', data.token);
      nav('/');
    } catch (e) { setError(e.response?.data?.msg || 'Something went wrong'); }
  };

  const set = k => e => setForm({ ...form, [k]: e.target.value });

  return (
    <div style={{ maxWidth: 320, margin: '80px auto' }}>
      <h2>{mode === 'login' ? 'Login' : 'Register'}</h2>
      {mode === 'register' && <input placeholder="Name" onChange={set('name')} />}
      <input placeholder="Email" onChange={set('email')} />
      <input placeholder="Password" type="password" onChange={set('password')} />
      <button onClick={submit}>Submit</button>
      <p style={{ color: 'red' }}>{error}</p>
      <Link to={mode === 'login' ? '/register' : '/login'}>
        {mode === 'login' ? 'Create account' : 'Have an account?'}
      </Link>
    </div>
  );
}
```

src/pages/Dashboard.jsx

```
import { useEffect, useState } from 'react';
import api from '../api';

export default function Dashboard() {
  const [list, setList] = useState([]);
  const [form, setForm] = useState({ title: '', amount: '', type: 'expense', category: '' });

  const load = async () => setList((await api.get('/transactions')).data);
  useEffect(() => { load(); }, []);

  const add = async () => {
    await api.post('/transactions', { ...form, amount: Number(form.amount) });
    setForm({ title: '', amount: '', type: 'expense', category: '' });
    load();
  };
  const remove = async id => { await api.delete(`/transactions/${id}`); load(); };

  const income = list.filter(t => t.type === 'income').reduce((s, t) => s + t.amount, 0);
  const expense = list.filter(t => t.type === 'expense').reduce((s, t) => s + t.amount, 0);

  return (
    <div style={{ maxWidth: 600, margin: '40px auto' }}>
      <h2>Balance: ₹{income - expense}</h2>
      <p>Income ₹{income} | Expense ₹{expense}</p>
```

```
      <input placeholder="Title" value={form.title} onChange={e => setForm({ ...form, title: e.target
      <input placeholder="Amount" type="number" value={form.amount} onChange={e => setForm({ ...form,
      <input placeholder="Category" value={form.category} onChange={e => setForm({ ...form, category:
      <select value={form.type} onChange={e => setForm({ ...form, type: e.target.value })}>
        <option value="expense">Expense</option>
        <option value="income">Income</option>
      </select>
      <button onClick={add}>Add</button>

      <ul>
        {list.map(t => (
          <li key={t._id}>
            {t.title} ({t.category}): {t.type === 'income' ? '+' : '-'}₹{t.amount}
            <button onClick={() => remove(t._id)}>x</button>
          </li>
        ))}
      </ul>
      <button onClick={() => { localStorage.removeItem('token'); location.href = '/login'; }}>Logout<
    </div>
  );
}
```

5. Upgrades (only if time allows)

1. Edit transaction (`PUT /:id`)
2. Pie chart by category (`npm i chart.js react-chartjs-2`)
3. Filter by month
4. Basic styling with plain CSS or Tailwind
5. Input validation and nicer error messages

6. Deployment

- **Database:** MongoDB Atlas (already set up)
- **Backend:** Render or Railway. Set env vars `MONGO_URI` and `JWT_SECRET`, start command `node server.js`
- **Frontend:** Vercel. Change `baseURL` in `api.js` to your deployed backend URL (better: use `import.meta.env.VITE_API_URL`)

7. Common problems

- **CORS error:** make sure `app.use(cors())` is before your routes.
- **Cannot connect to Mongo:** check the Atlas IP allowlist and that your password has no special characters (or URL-encode it).
- **401 errors:** confirm the token is saved in localStorage and the header is `Bearer <token>`.
- **Blank page after deploy:** check the browser console, usually a wrong API URL.

8. Viva cheat-sheet

- **Why MERN?** One language (JavaScript) across the whole stack, JSON everywhere

- **How does auth work?** Password hashed with bcrypt; on login the server signs a JWT; the client sends it in the `Authorization` header; middleware verifies it.
- **Why MongoDB?** Flexible documents, easy to map to JavaScript objects via Mongoose.
- **What is middleware?** A function that runs between request and route handler (we use it for auth).
- **Where is data stored per user?** Each transaction has a `user` reference, and every query filters by it.

9. Git checklist

```
git init
git add . && git commit -m "backend auth done"
```

Commit after each milestone (backend auth, transactions API, frontend login, dashboard). It makes a good project history for your submission.